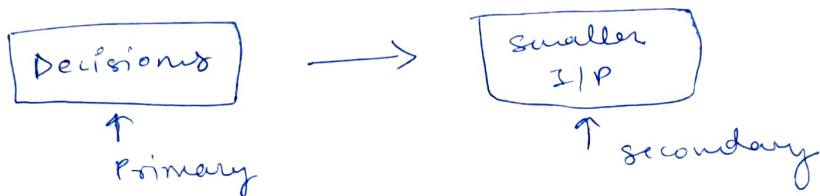


Recursion

- Make I/P smaller? But why?
- Recursion - Decision Space
- Recursive Tree - Soul of Recursion
- 2 steps to solve any Recursive problem

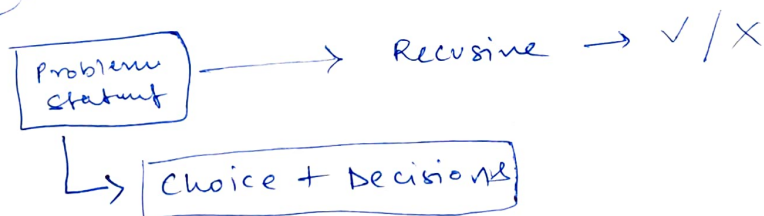
★ Make I/P smaller?

No, It becomes smaller automatically.



★ Recursion → Decision Space

Identification



★ Recursive Tree → Imp

PS →



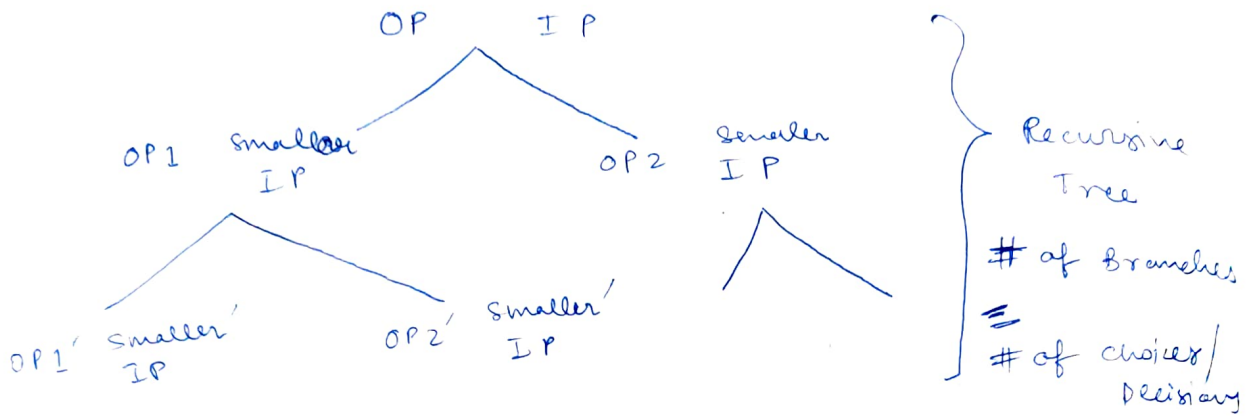
RT

→ Code ~~called~~ k



Recursive Trees ??

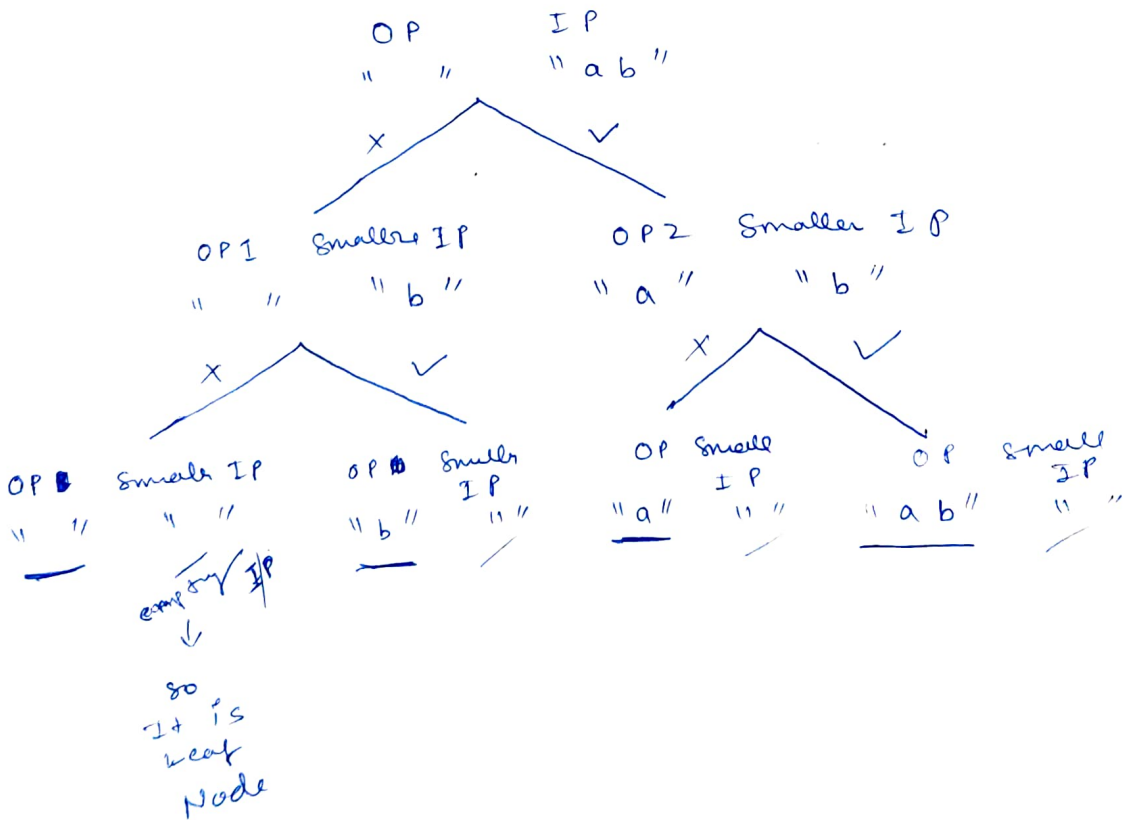
* IP-OP Method :



Example :

Input : "ab"

Find all subsets of IP "ab"



★ 2 Steps to Solve any Recursive Problem

1. Design a Recursive tree ✓ Imp

2. Fuck the Problem / Write the Code

Recursion Problems

IP-OP method

1. Print 1 to N / N to 1.
2. Sort an array → Sort a Stack
3. Delete middle Element in a Stack.
4. Remove Duplicates from a String.
5. Count the # of occurrences
6. Subset
7. Permutations (a) spaces (variations)
(b) case change
(c) and many more

Extended IP-OP method

1. Binary String with $\#1s > \#0s$
2. Generate Balanced Parentheses.

8. Josephus Problem / Execution in Circle

Base Condⁿ - Hypothesis - Induction

- Methods To Think for a Recursive Problem
- Why Making I/P Smaller is Popular / Imp.

★ 4 Approaches

1. Recursive Tree - IP-OP method (Decision)
2. Base Condⁿ - Induction - Hypothesis $\leftarrow$ (making IP smaller)
IBH
3. Choice Diagram
4. X later

* Base Condⁿ



Smallest + Valid IP

Invalid IP

IBH method (useful)



Tree

Linked list

* Recursion

Simple $\rightarrow$ IBH

Med $\rightarrow$  RT

Hard $\rightarrow$ choice Diagram

Sort an Array Using Recursion

```
sort ( ) {
}
}
```

* Design Hypothesis

⇒

4	2	9	6	X
---	---	---	---	---

 ↓ Remove last element and sort smaller IP

sort (

4	2	9	6
---	---	---	---

);

no. of elements

Invalid	0	n-2	n-1	n
---------	---	-----	-----	---

* Base condⁿ:

```
if (v.size() == 1)
    return;
```

* Induction:

1	5	0	2
---	---	---	---

sort (

1	5	0
---	---	---

)

2

0	1	5
---	---	---

 ←

2

 insert at right pos

0	1	2	5
---	---	---	---

* insert ()

```
insert (vector, temp)
    ↓
    

|   |   |   |
|---|---|---|
| 0 | 1 | 5 |
|---|---|---|



|   |
|---|
| 2 |
|---|


```

→ * Design Hypothesis

0	1	X
---	---	---

↓

0	1
---	---

 ←

2

 v.push-back

* Base condⁿ:

```
if (v.size() == 0 || v[v.size()-1] >= temp) {
    v.push-back(temp);
    return;
}
```

0	1	5
---	---	---

 ←

6

 push-back

* Induction:

insert () {

BC →

int val = v[v.size()-1]

v.pop-back();

insert (v, temp); insert (

0	1
---	---

,

2

)

v.push-back (~~temp~~ val);

0	1	2
---	---	---

 ←

5

 push-back

code P.T.O

Sort an Array

```
class Solution {
public:
    void insert(vector<int> &v, int temp){
        if(v.size() == 0 || v[v.size() - 1] <= temp){
            v.push_back(temp);
            return;
        }

        int val = v[v.size()-1];
        v.pop_back();

        insert(v, temp);

        v.push_back(val);
    }
    void sortArr(vector<int> &arr) {
        if(arr.size() == 1)
            return;

        int val = arr[arr.size()-1];
        arr.pop_back();

        sortArr(arr);
        insert(arr, val);
    }
};
```

Sort a Stack

```
class Solution {
public:
    void insert(stack<int> &s, int temp) {
        if(s.size() == 0 || s.top() <= temp){
            s.push(temp);
            return;
        }

        int val = s.top();
        s.pop();

        insert(s, temp);
        s.push(val);
    }
    void sortStack(stack<int> &s) {
        if(s.size() == 1)
            return;

        int val = s.top();
        s.pop();

        sortStack(s);
        insert(s, val);
    }
};
```

Delete Mid of a Stack

```
class Solution {
public:
    void solve(stack<int>& s, int k){
        if(s.size() == k){
            s.pop();
            return;
        }

        int val = s.top();
        s.pop();

        solve(s, k);

        s.push(val);
    }
    // Function to delete middle element of a stack.
    void deleteMid(stack<int>& s) {

        int k = floor((s.size()+1)/2);

        solve(s, k);
    }
};
```


Reverse a Stack

```
class Solution {
public:
    void insert(stack<int> &s, int temp){
        if(s.size() == 0){
            s.push(temp);
            return;
        }

        int val = s.top();
        s.pop();

        insert(s, temp);

        s.push(val);
    }
    void reverse(stack<int> &s) {
        if(s.size() == 1){
            return;
        }

        int val = s.top();
        s.pop();

        reverse(s);

        insert(s, val);
    }
};
```

K-th Symbol in Grammar

Code | C++

```
class Solution {
public:
    int kthGrammar(int n, int k) {
        if(n == 1 && k == 1)
            return 0;

        int mid = pow(2, n-1) / 2;

        if(k <= mid)
            return kthGrammar(n-1, k);
        else
            return !kthGrammar(n-1, k-mid);
    }
};
```

Tower of Hanoi

```
recursion > C++ tower_of_hanoi.cpp > main()
1  #include<iostream>
2  #include<math.h>
3  using namespace std;
4
5  void tohcal(int n, int source, int help, int dest) {
6
7      if(n == 1){
8          cout<<"move disk "<<n<<" from rod "<<source<<" to rod "<<dest<<endl;
9          return;
10     }
11
12     tohcal(n-1, source, dest, help);
13
14     cout<<"move disk "<<n<<" from rod "<<source<<" to rod "<<dest<<endl;
15
16     tohcal(n-1, help, source, dest);
17 }
18
19 int main() {
20
21     int n;
22     cin>>n;
23
24     int source = 1, help = 2, dest = 3;
25
26     tohcal(n, source, help, dest);
27
28     cout<<pow(2,n) - 1<<endl;
29
30     return 0;
31 }
```

```
3
move disk 1 from rod 1 to rod 3
move disk 2 from rod 1 to rod 2
move disk 1 from rod 3 to rod 2
move disk 3 from rod 1 to rod 3
move disk 1 from rod 2 to rod 1
move disk 2 from rod 2 to rod 3
move disk 1 from rod 1 to rod 3
7
```

Josephus Problem

```
void solve(int index, int k, vector<int>& arr, int& ans){
    if(arr.size() == 1){
        ans = arr[0];
        return;
    }

    index = (index + k) % arr.size();
    arr.erase(arr.begin() + index);

    solve(index, k, arr, ans);
    return;
}

int safePos(int n, int k) {
    vector<int> arr(n);
    for(int i=0 ; i<n ; i++){
        arr[i] = i+1;
    }
    k--;
    int ans = -1;
    solve(0, k, arr, ans);

    return ans;
}
```

IP-OP Method

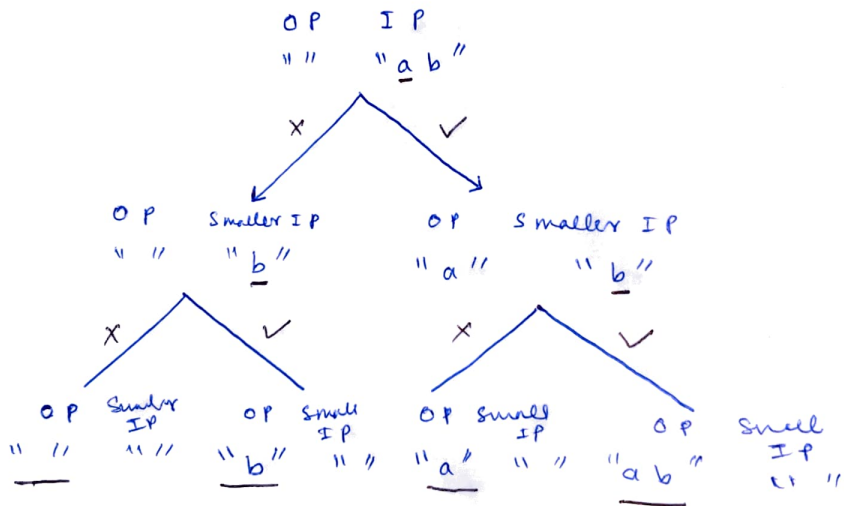
- Subset
- Subset Duplicates + other variations
- Permutation with space
- Permutation with case change
- Josephus problem
- Print space string.

Subsequences / Subset / Powersets

ip = "ab"

op = ""

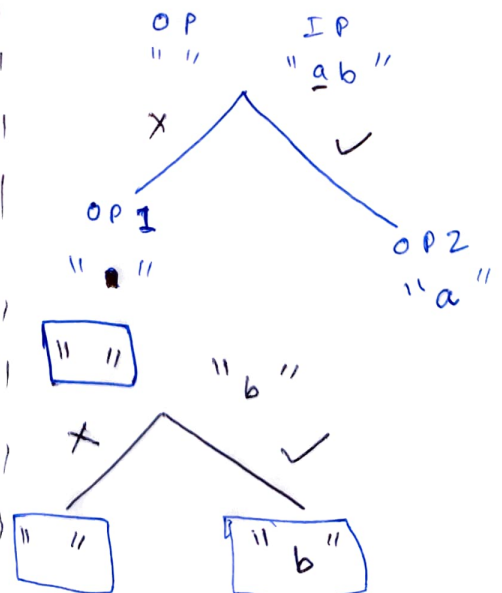
IP-OP RT



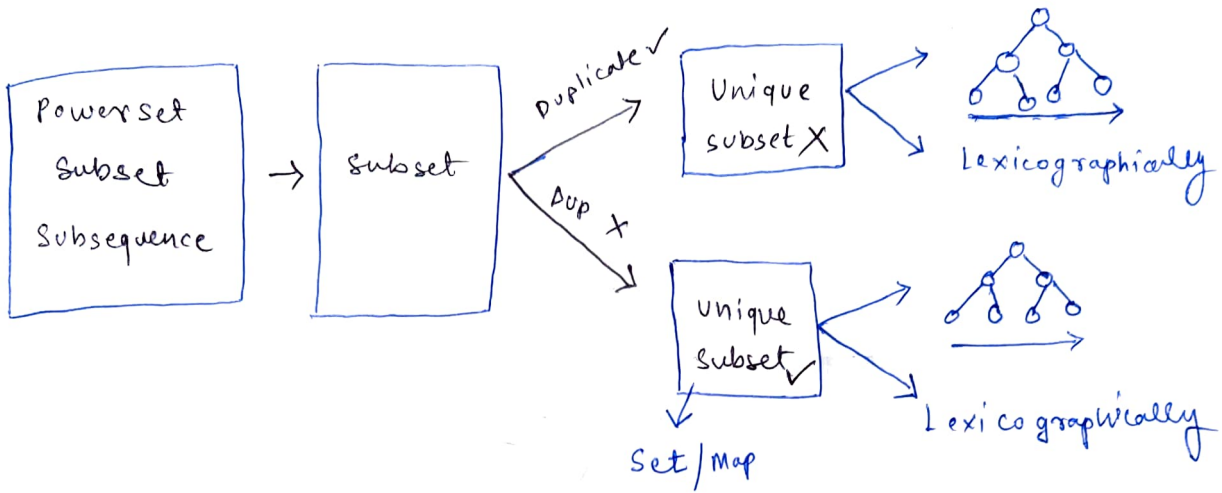
Code

```
void solve(string ip, string op) {
    if (ip.length() == 0) {
        cout << op << " ";
        return;
    }
    string op1 = op;
    string op2 = op;
    op2.push_back(ip[0]);
    ip.erase(ip.begin() + 0);
    solve(ip, op1);
    solve(ip, op2);
    return;
}
```

RT



Point Unique Subsets and Variations

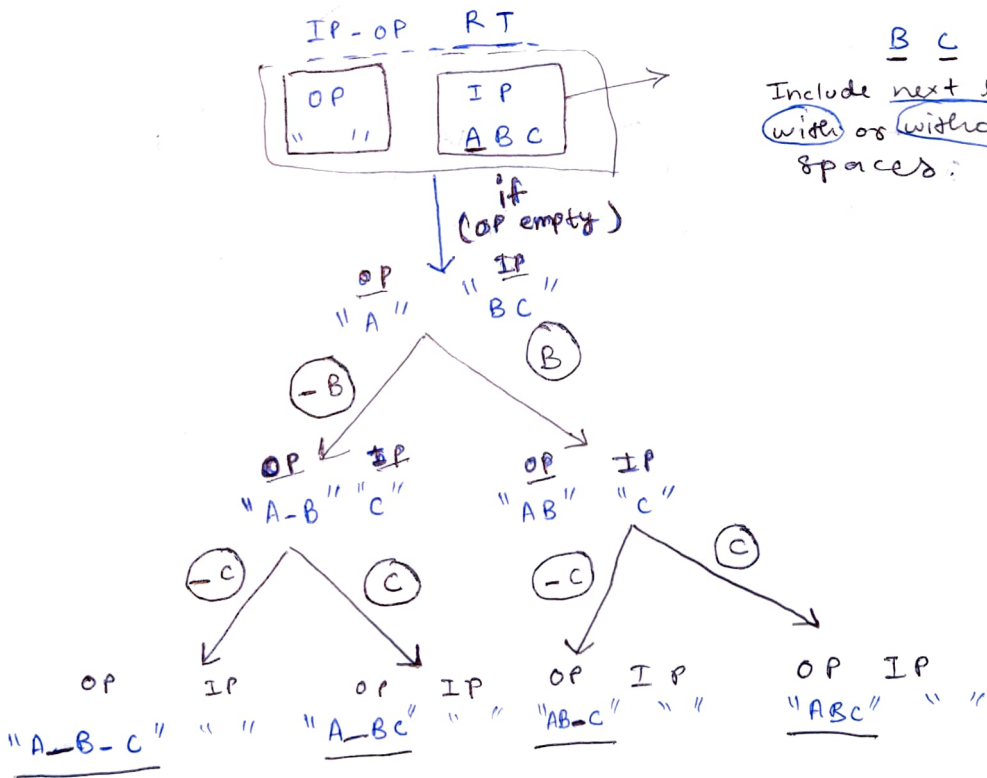


Permutation with Spaces

Problem Statement :

IP: ABC

OP: A-B-C, A-BC, AB-C, ABC



Include next letter
with or without
spaces:

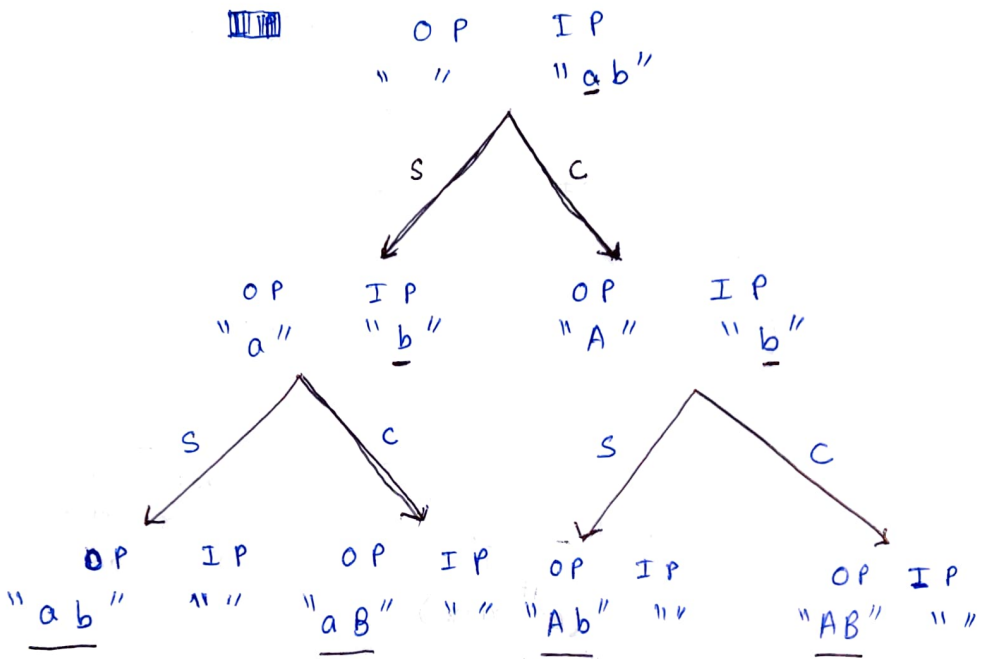
Permutation with Case Change

Problem Statement :

IP : ab

OP : ab aB AB AB

IP-OP RT



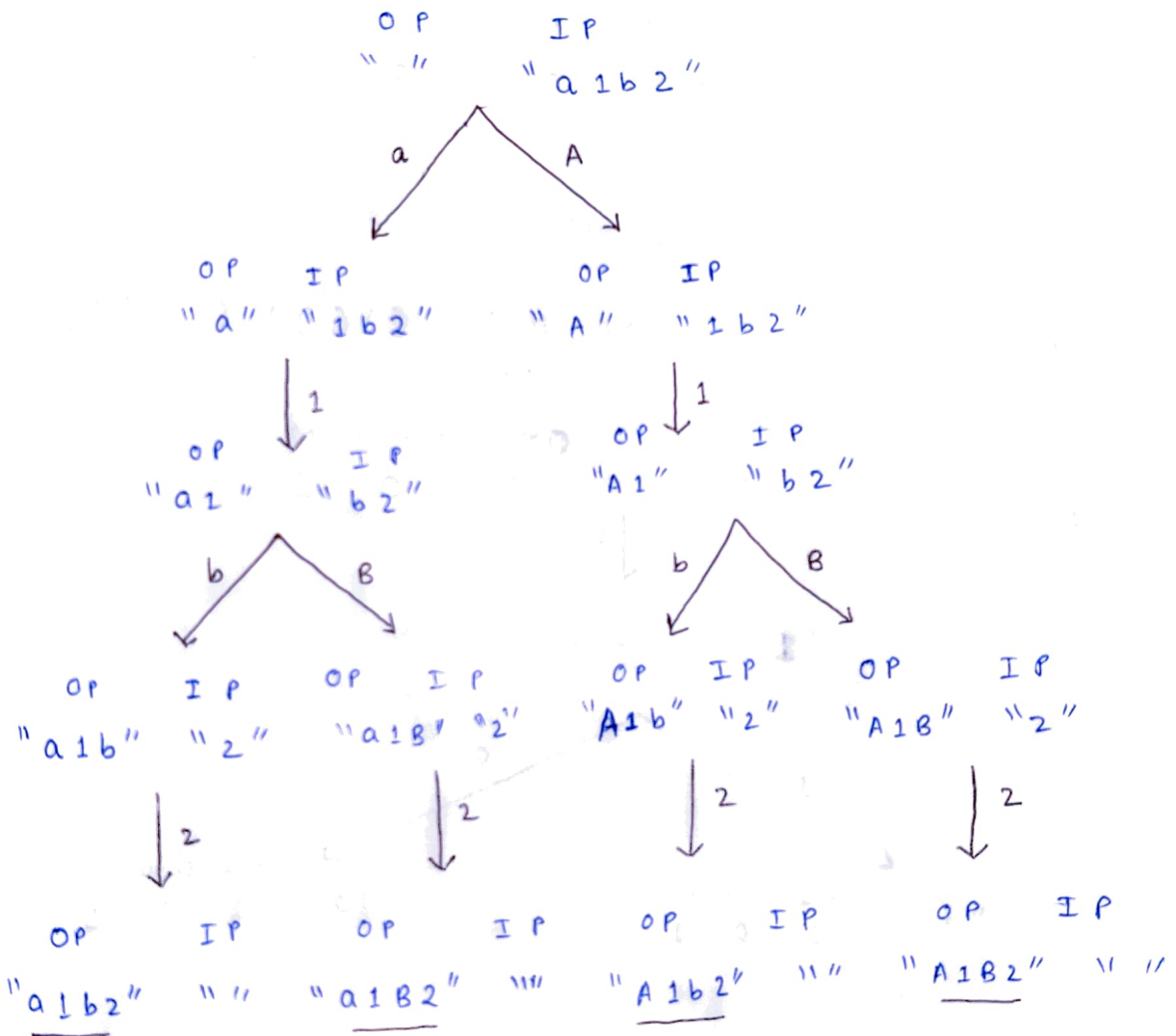
Letter Case Permutation

Problem Statement:

IP : a1b2

OP : a1b2, a1B2, A1b2, A1B2

IP-OP RT



Subsets

```
void solve(string ip, string op, vector<string>& ans){
    if(ip.length() == 0){
        if(op.length() != 0)
            ans.push_back(op);
        return;
    }

    string op1 = op;
    string op2 = op;
    op2.push_back(ip[0]);

    ip.erase(ip.begin() + 0);

    solve(ip, op1, ans);
    solve(ip, op2, ans);
    return;
}

vector<string> AllPossibleStrings(string s) {
    vector<string> ans;
    string ip = s;
    string op = "";

    solve(ip, op, ans);
    sort(ans.begin(), ans.end());

    return ans;
}
```

Unique Subsets

```
void solve(vector<int> ip, vector<int> op, vector<vector<int>>& ans){
    if(ip.size() == 0){
        sort(op.begin(), op.end());
        ans.push_back(op);
        return;
    }

    vector<int> op1 = op;
    vector<int> op2 = op;
    op2.push_back(ip[0]);
    ip.erase(ip.begin() + 0);

    solve(ip, op1, ans);
    solve(ip, op2, ans);
    return;
}

// Function to find all possible unique subsets.
vector<vector<int>> AllSubsets(vector<int> arr, int n) {
    vector<vector<int>> ans;
    vector<int> ip = arr;
    vector<int> op;
    // call the function to get subsets
    solve(ip, op, ans);
    // store in set to get uniques
    set<vector<int>> st;
    for(int i=0 ; i<ans.size() ; i++){
        st.insert(ans[i]);
    }
    // convert back to vector array of arrays and sort for lexicography
    ans.clear();
    for(auto it : st){
        ans.push_back(it);
    }
    sort(ans.begin(), ans.end());
    return ans;
}
```

Permutation with Spaces

```
void solve(string ip, string op, vector<string>& ans){
    if(ip.size() == 0){
        ans.push_back(op);
        return;
    }

    string op1 = op;
    string op2 = op;
    op1.push_back(' ');
    op1.push_back(ip[0]);
    op2.push_back(ip[0]);
    ip.erase(ip.begin() + 0);

    solve(ip, op1, ans);
    solve(ip, op2, ans);
    return;
}

vector<string> permutation(string s) {
    vector<string> ans;
    string ip = s;
    string op = "";

    op.push_back(ip[0]);
    ip.erase(ip.begin() + 0);

    solve(ip, op, ans);

    return ans;
}
```

Permutation with Case Change

```
// CPP code to print all permutations with respect to cases
#include <bits/stdc++.h>
using namespace std;

// Function to generate permutations
void permute(string ip, string op) {
    // base case
    if (ip.size() == 0) {
        cout << op << " ";
        return;
    }
    // pick lower and uppercase
    char ch = tolower(ip[0]);
    char ch2 = toupper(ip[0]);
    ip = ip.substr(1);

    permute(ip, op + ch);
    permute(ip, op + ch2);
}

// Driver code
int main() {
    string ip = "ab";
    permute(ip, "");
    return 0;
}
```



ab aB Ab AB

Unique K-Number Sum

Unique K-Number Sum



Difficulty: **Medium**

Accuracy: **74.43%**

Submissions: **7K+**

Points: **4**

Given two integers **n** and **k**, the task is to find all valid combinations of **k** numbers that adds up to **n** based on the following conditions:

- **Only** numbers from the range [1, 9] used.
- Each number can only be used **at most** once.

Note: You can return the combinations in any order, the driver code will print them in sorted order.

Examples:

Input: n = 9, k = 3

Output: [[1, 2, 6], [1, 3, 5], [2, 3, 4]]

Explanation: There are three valid combinations of 3 numbers that sum to 9: [1, 2, 6], [1, 3, 5] and [2, 3, 4].

Input: n = 3, k = 3

Output: []

Explanation: It is not possible to pick 3 distinct numbers from 1 to 9 that sum to 3, so no valid combinations exist.

Constraints:

$$1 \leq n \leq 50$$

$$1 \leq k \leq 9$$

[Try more examples](#)

Expected Complexities

Time Complexity: $O(k * 2^9)$

Auxiliary Space: $O(k)$

```

void solve(vector<int> ip, vector<int> op, int n, int k, vector<vector<int>>& ans){
    if(ip.size() == 0){
        if(op.size() == k && accumulate(op.begin(), op.end(), 0) == n)
            ans.push_back(op);
        return;
    }

    vector<int> op1 = op;
    vector<int> op2 = op;
    op2.push_back(ip[0]);
    ip.erase(ip.begin() + 0);

    solve(ip, op1, n, k, ans);
    solve(ip, op2, n, k, ans);
    return;
}

vector<vector<int>> combinationSum(int n, int k) {
    vector<vector<int>> ans;
    vector<int> ip = {1,2,3,4,5,6,7,8,9};
    vector<int> op;

    solve(ip, op, n, k, ans);

    return ans;
}

```

Letter Case Permutation

```
void solve(string ip, string op, vector<string>& ans){
    if(ip.size() == 0){
        ans.push_back(op);
        return;
    }

    if(isalpha(ip[0])){
        string op1 = op;
        string op2 = op;
        op1.push_back(tolower(ip[0]));
        op2.push_back(toupper(ip[0]));
        ip.erase(ip.begin() + 0);

        solve(ip, op1, ans);
        solve(ip, op2, ans);
    }
    else{
        string op1 = op;
        op1.push_back(ip[0]);
        ip.erase(ip.begin() + 0);

        solve(ip, op1, ans);
    }

    return;
}

vector<string> letterCasePermutation(string s) {
    vector<string> ans;
    string ip = s;
    string op = "";

    solve(ip, op, ans);

    return ans;
}
```

Extended IP-OP Method

In this the main difference is that

IP OP



different datatypes

- Generate all Balanced Parentheses
- Print N-bit binary nos having more 1's than 0's for any prefix
- Josephus Problem

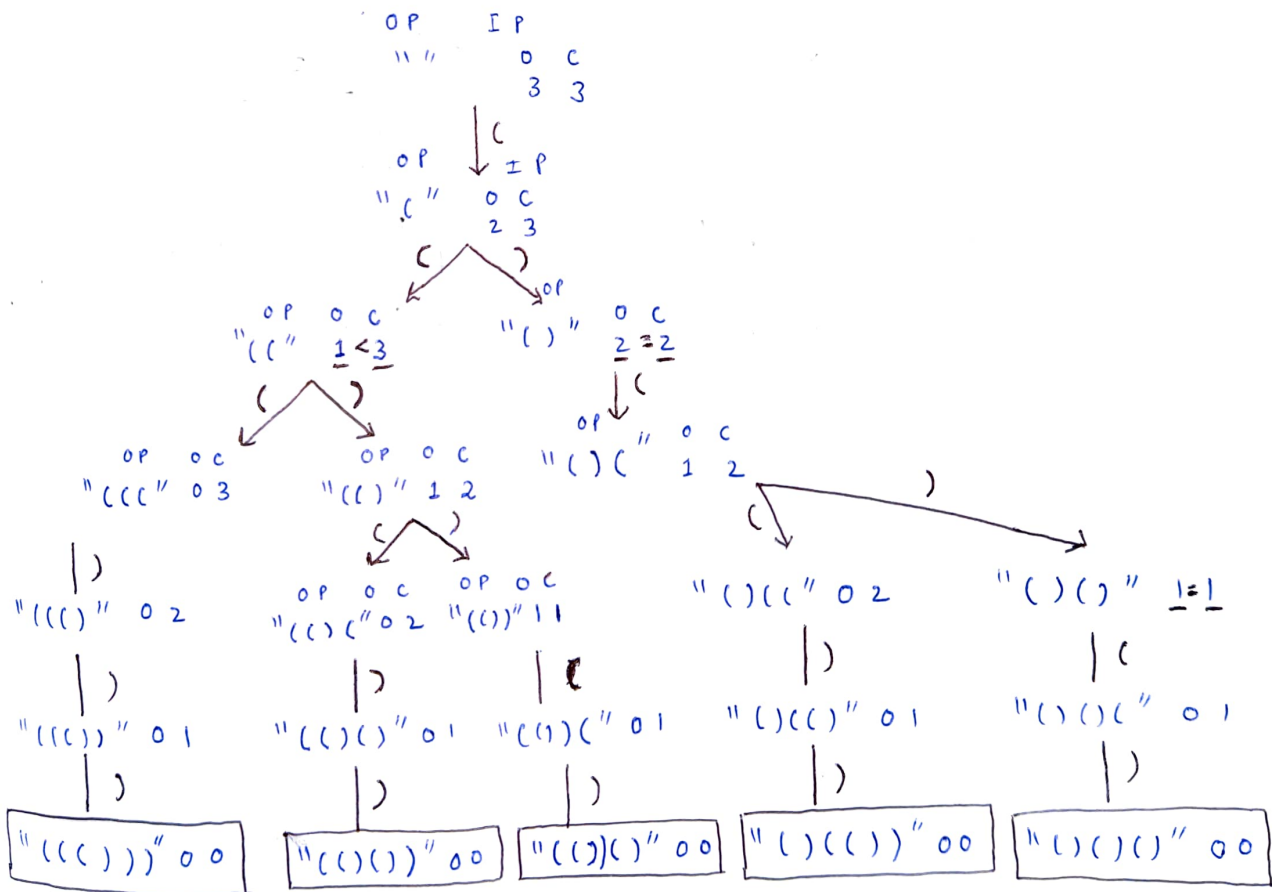
Generate all Balanced Parentheses

problem statement:

IP : 3

OP : ((())), ((()())), (()()), ()(()), ()()()

IP-OP RT



Print N-bit Binary Numbers

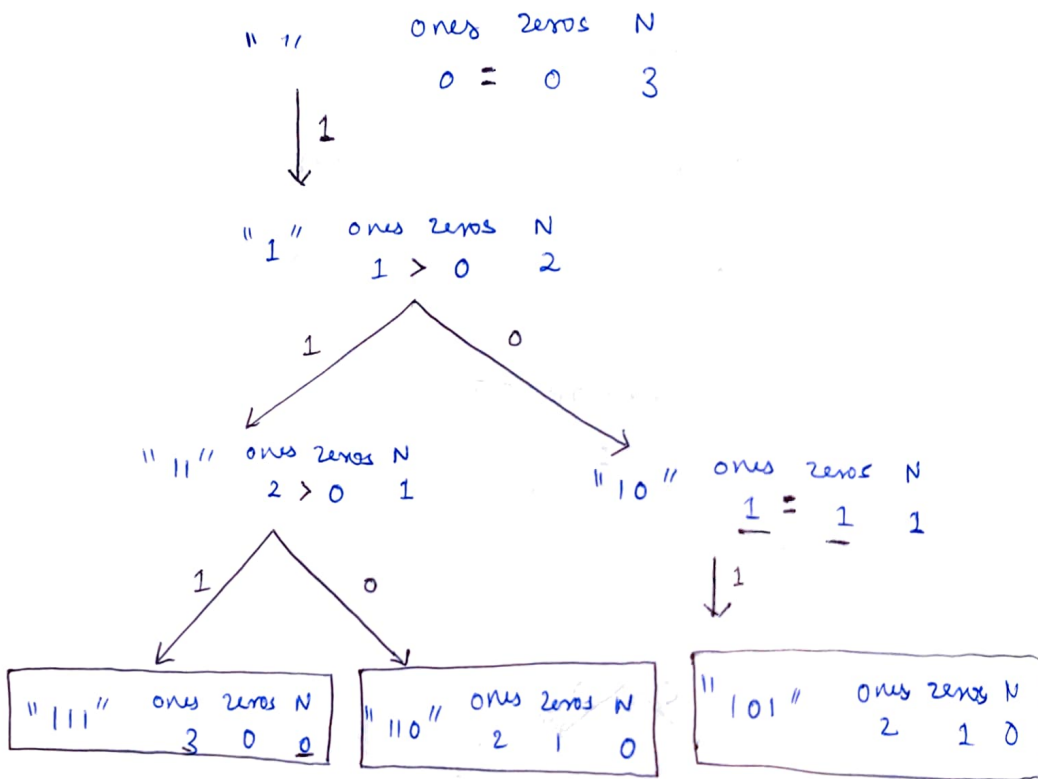
Having More 1's than 0's
for any prefix

Problem Statement :

IP : 3

OP : 111, 110, 101

IP-OP RT



Generate Parentheses

```
void solve(int open, int close, string op, vector<string>& v){
    if(open == 0 && close == 0){
        v.push_back(op);
        return;
    }

    if(open != 0){
        string op1 = op;
        op1.push_back('(');
        solve(open - 1, close, op1, v);
    }
    if(close > open){
        string op1 = op;
        op1.push_back(')');
        solve(open, close - 1, op1, v);
    }
    return;
}

vector<string> generateParenthesis(int n) {
    vector<string> ans;
    int open = n;
    int close = n;
    string op = "";

    solve(open, close, op, ans);

    return ans;
}
```

Print N-bit binary numbers having more 1s than 0s

```
void solve(int ones, int zeros, string op, int n, vector<string>& ans){
    if(n == 0){
        ans.push_back(op);
        return;
    }

    string op1 = op;
    op1.push_back('1');
    solve(ones + 1, zeros, op1, n - 1, ans);

    if(ones > zeros){
        op1 = op;
        op1.push_back('0');
        solve(ones, zeros + 1, op1, n - 1, ans);
    }
    return;
}

vector<string> NBitBinary(int n) {
    vector<string> ans;
    int ones = 0;
    int zeros = 0;
    string op = "";

    solve(ones, zeros, op, n, ans);

    return ans;
}
```

Alternate-code

```
void solve(int ones, int zeros, string op, int n, vector<string>& ans){
    if(n == 0){
        ans.push_back(op);
        return;
    }

    solve(ones + 1, zeros, op + '1', n - 1, ans);
    if(ones > zeros)
        solve(ones, zeros + 1, op + '0', n - 1, ans);

    return;
}

vector<string> NBitBinary(int n) {
    vector<string> ans;
    int ones = 0;
    int zeros = 0;
    string op = "";

    solve(ones, zeros, op, n, ans);

    return ans;
}
```